Robot Control - Project 3 - Report

Jakub Pniewski 459 481

January 2026

1 Kalman filter

I implemented a linear Kalman Filter using a Constant Velocity motion model. This model assumes that the drone maintains a constant velocity during the time step Δt, and acceleration is modeled as white noise (Discrete White Noise Acceleration - DWNA). This is a big simplification of physics but it works well,

The state vector consists of 3D positions and 3D velocities:

$$x = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^T$$

. I propagate the state using the transition matrix F. The new position is the old position plus velocity multiplied by time.

$$F = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Since the Aruco detection provides only the 3D position of the drone, the measurement matrix H maps the state space to the measurement space by selecting only the position components:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

To determine the Measurement Noise Covariance R, I collected data while the drone was hovering and calculated the covariance of the error between the Aruco measurements and the ground truth. This captures the sensor noise:

$$R = \begin{bmatrix} \sigma_{xx}^2 & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy}^2 & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz}^2 \end{bmatrix} \approx \begin{bmatrix} 3.6 \times 10^{-5} & 5.5 \times 10^{-7} & 1.6 \times 10^{-5} \\ 5.5 \times 10^{-7} & 2.4 \times 10^{-6} & -2.2 \times 10^{-7} \\ 1.6 \times 10^{-5} & -2.2 \times 10^{-7} & 1.0 \times 10^{-5} \end{bmatrix}$$

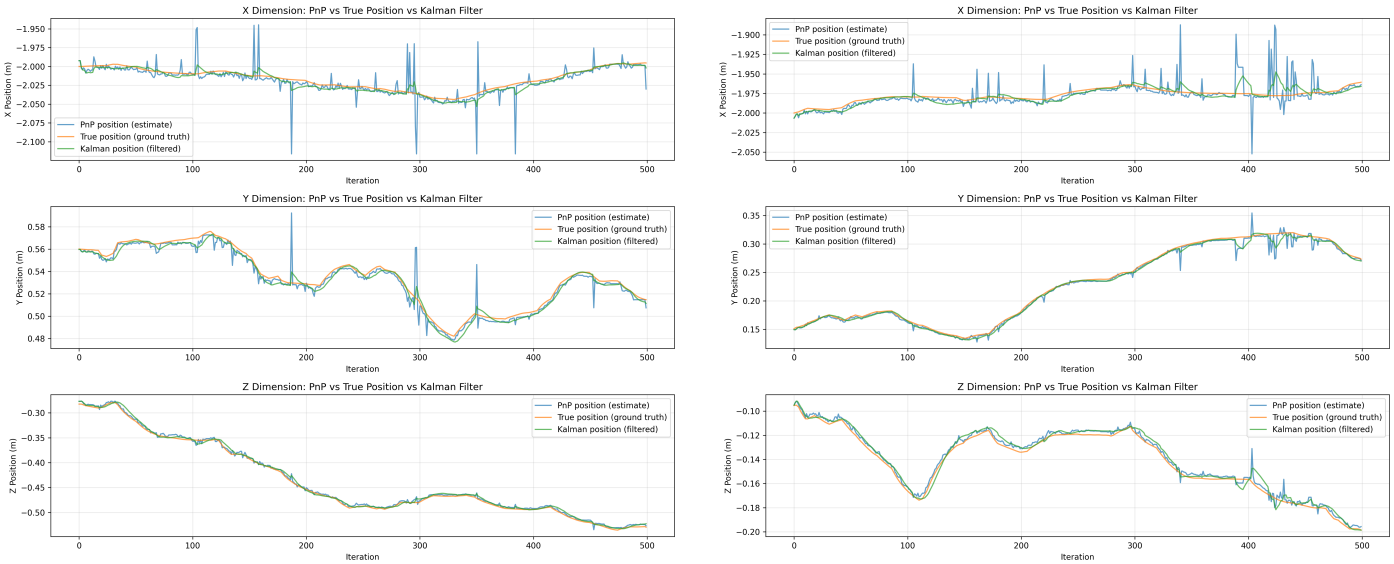
For the Process Noise Covariance Q, I used the Discrete White Noise Acceleration formulation. This assumes that the process noise enters the system through acceleration. The matrix links position uncertainty to velocity uncertainty for each axis. σ_q is a tunable hyperparameter representing the variance of the acceleration:

$$Q = \sigma_q \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & 0 & \frac{\Delta t^3}{2} & 0 & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & 0 & \frac{\Delta t^4}{4} & 0 & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & 0 & \Delta t^2 & 0 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & 0 & \Delta t^2 & 0 \\ 0 & 0 & \frac{\Delta t^3}{2} & 0 & 0 & \Delta t^2 \end{bmatrix}$$

The filter is initialized with zero state $x_0 = \vec{0}$, but with a high initial covariance $P_0 = 100 \cdot I_6$ to reflect that the initial position estimate is completely uncertain, allowing the filter to converge quickly to the first measurements.

2 Plots

The results are very good. Localization relative to the first gate yields:

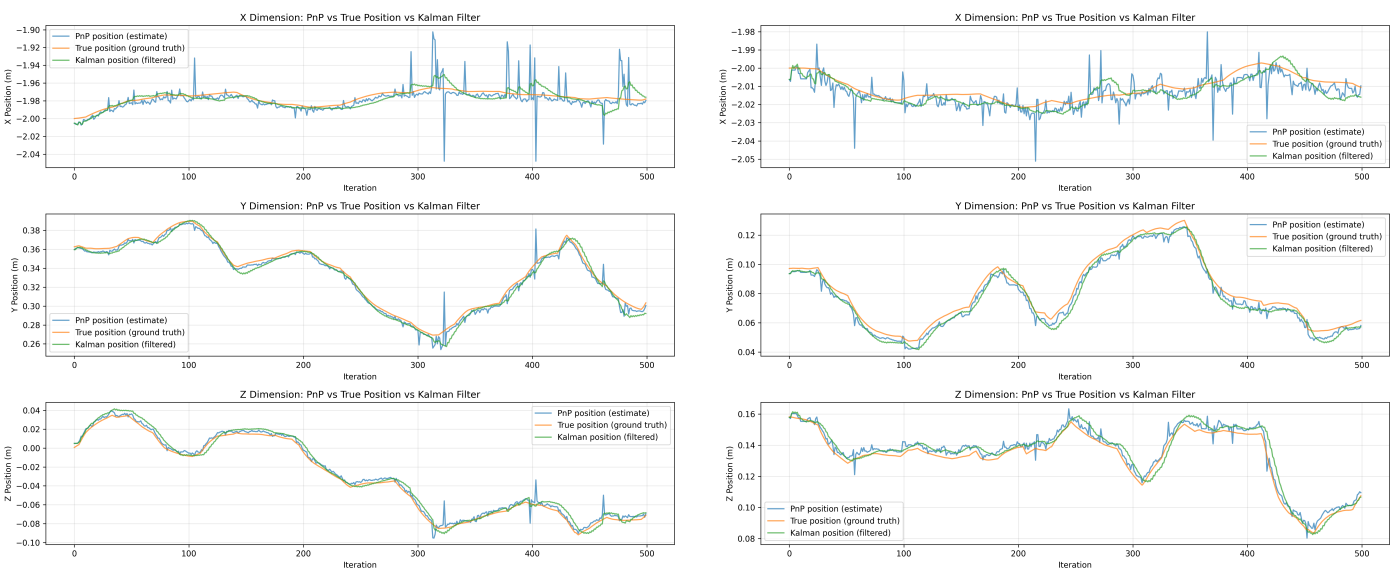


(a) Gates straight

(b) Gates rotated

Figure 1: Results

The Kalman predictions effectively smooth out noise spikes, providing a more accurate approximation of the true position compared to raw observations. Additionally, updating the filter only every second frame (alternating with blind prediction steps) also provides robust results:



(a) Gates straight

(b) Gates rotated

Figure 2: Observation every second timestep

3 Insight from this project

I found out that the PnP solver is partially robust to coordinate errors. Even when ArUco corner coordinates were incorrectly inverted along the MuJoCo y-axis, the solver produced plausible results in approximately half of the time, as shown below.

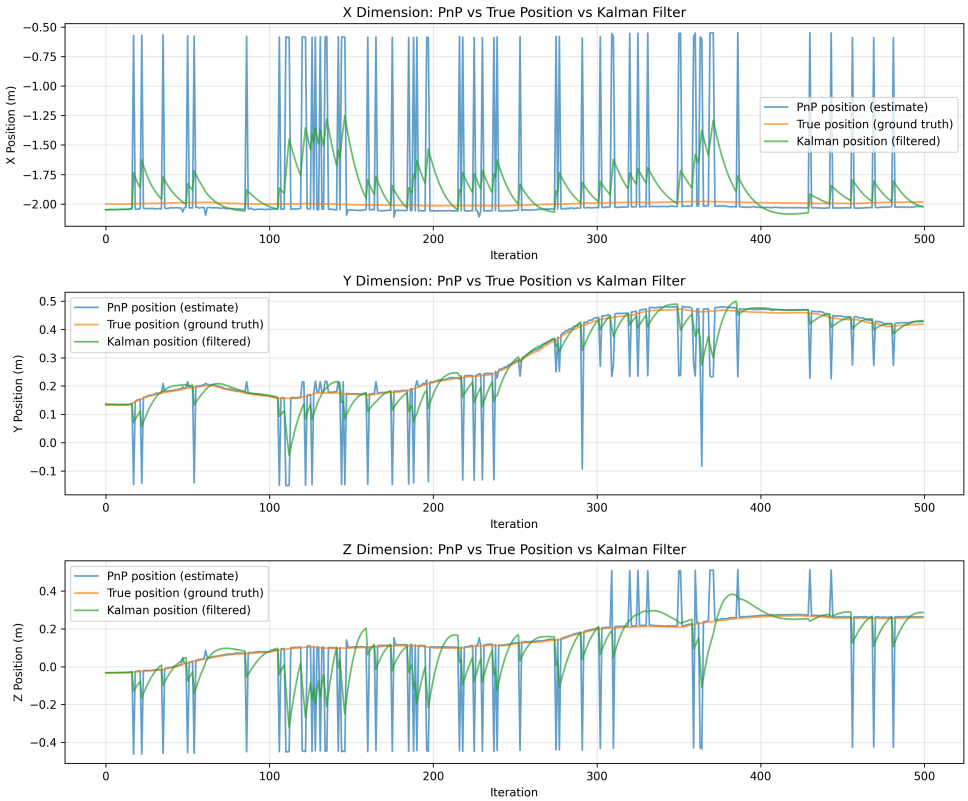


Figure 3: Results with coordinate error